

Chapter 8 结构体



本章内容

- 结构体（结构structure）类型的定义
- 结构体变量、结构体数组
- 结构体变量、结构体数组与指针、函数的关系
- 位运算
- 枚举常量

思考一个问题

- 在程序里表示一个人（姓名、年龄、性别、.....），怎么表示？
- 想表示多个人呢？
- 如何用计算机程序实现下述表格的管理？

表 某学校学生成绩管理表

学号	姓名	性别	入学时间	计算机原理	英 语	数 学	音 乐
1	令狐冲	M	1999	90	83	72	82
2	林平之	M	1999	78	92	88	78
3	岳灵珊	F	1999	89	72	98	66
4	任莹莹	F	1999	78	95	87	90
5							
6							

数组的解决方法

```
int    studentId[30];      /* 最多可以管理30个学生， 每个学生的学号用数组的下标表示*/
char   studentName[30][10]; /*30个学生的姓名*/
char   studentSex[30];     /*30个学生的性别*/
int    timeOfEnter[30];    /*30个学生入学时间*/
int    scoreComputer[30];  /*30个学生计算机原理课的成绩*/
int    scoreEnglish[30];  /*30个学生英语课的成绩*/
int    scoreMath[30];      /*30个学生数学课的成绩*/
int    scoreMusic[30];     /*30个学生音乐课的成绩*/
```

```
int    studentId[30] = {1,2,3,4,5,6};
char   studentName[30][10] = {"令狐冲", "林平之", "岳灵珊", "任莹莹"};
char   studentSex[30] = {'M', 'M', 'F', 'F'};
int    timeOfEnter[30] = {1999, 1999, 1999, 1999};
int    scoreComputer[30] = {90, 78, 89, 78};
int    scoreEnglish[30] = {83, 92, 72, 95};
int    scoreMath[30] = {72, 88, 98, 87};
int    scoreMusic[30] = {82, 78, 66, 90};
```


数组的解决方法

- 数据的内存管理方式

1
2
3
4
.....

令狐冲
林平之
岳灵珊
任莹莹
.....

M
M
F
F
.....

1999
1999
1999
1999
.....

90
78
89
78
.....

83
92
72
95
.....

72
88
98
87
.....

82
78
66
90
.....

数组的解决方法

- 分配内存不集中，寻址效率不高
- 对数组进行赋初值时，容易发生错位
- 结构显得比较零散，不容易管理

希望的内存分配图

1
令狐冲
男
1999
90
83
72
82

2
林平之
男
1999
78
92
88
78

3
岳灵珊
女
1999
89
72
98
66

4
任莹莹
女
1999
78
95
87
90

结构体的解决方法

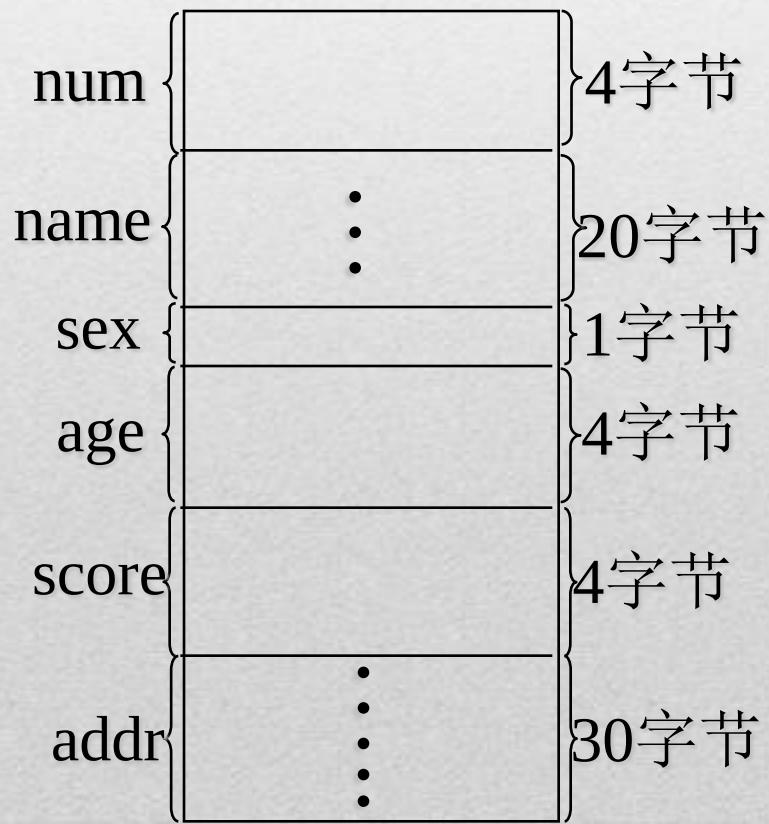
```
struct STUDENT
{
    int    studentID;        /*每个学生的序号*/
    char   studentName[10]; /*每个学生的姓名*/
    char   studentSex;       /*每个学生的性别*/
    int    timeOfEnter;      /*每个学生的入学时间*/
    int    scoreComputer;    /*每个学生的计算机原理成绩*/
    int    scoreEnglish;     /*每个学生的英语成绩*/
    int    scoreMath;        /*每个学生的数学成绩*/
    int    scoreMusic;       /*每个学生的音乐成绩*/
};
```

- `struct STUDENT` 是一个类型
- `struct STUDENT students[4];`
- `students[0].studentName`
`students[0].Sex`
 - 它们都是变量，一般称为结构的成员变量

8.1 结构体定义

- 结构体：
 - 把关系紧密且逻辑相关的多种不同类型的变量组织到统一的名字之下，也称复合数据类型
 - 这种类型的变量占用相邻的一段内存单元

```
struct    student
{
    int num;
    char name[20];
    char sex;
    int age;
    float score;
    char addr[30];
};
```



8.1 结构体定义

num	name	sex	age	score	addr
10010	LiFun	M	18	87.5	Beijing

structure definition形成一个样板，用于生成结构体变量。

构成结构体的变量称为结构体的**成员 (member)**，
也称**元素 (element)** 或**域 (field)**

一般形式：

struct 结构体名

```
{  
    类型关键字    成员名1;  
    类型关键字    成员名2  
    .....  
    类型关键字    成员名n  
};
```

```
struct student  
{  
    int num;  
    char name[20];  
    char sex;  
    int age;  
    float score;  
    char addr[30];  
};
```

8.1 结构体定义

- 结构体定义中可以包括指针及其他结构体

```
struct Card
{
    char *face;
    char *suit;
};
```

```
struct date
{
    int month;
    int day;
    int year;
};

struct student
{
    int num;
    char name[20];
    struct date birthday;
};
```

- 自引用结构体

```
struct Student
{
    int num;
    char name[10];
    float score;
    struct Student monitor; //错误
    struct Student *pmonitor; //正确
};
```

8.2 结构体变量的定义

✂ 先定义结构体类型再定义变量名

```
struct student    student1, student2;
```

✂ 在定义类型的同时定义变量

```
struct student
{
    int    num;
    char    name[20];
    char    sex;
    int    age;
    float    score;
    char    addr[30];
} student1, student2;
```

```
struct
{
    int    num;
    char    name[20];
    char    sex;
    int    age;
    float    score;
    char    addr[30];
} student1, student2;
```

✂ 直接定义结构体变量（不出现结构体名）

说明

- 结构体类型与结构体变量概念不同

- 类型:不分配内存;

- 变量:分配内存

- 类型:不能赋值、存取、运算; 变量:可以

- ❖ 结构体成员名与程序中变量名可相同, 不会混淆

- ❖ 结构体可嵌套

```
例 struct date
{   int month;
    int day;
    int year;
};
struct student
{   int num;
    char name[20];
    struct date birthday;
}stu;
```

num	name	birthday		
		month	day	year

```
例 struct student
{   int num;
    char name[20];
    struct date
    {   int month;
        int day;
        int year;
    }birthday;
}stu;
```

结构体变量的初始化

```
struct student
{
    int num;
    char name[20];
    char sex;
    int age;
    char addr[30];
};
struct student stu1={112,"Wang Lin",'M',19, "200 Beijing Road"};
```

```
struct student
{
    int num;
    char name[20];
    char sex;
    int age;
    char addr[30];
}stu1={112,"Wang Lin",'M',19, "200 Beijing Road"};
```

结构体变量的引用

- 结构体变量不能整体引用,只能引用变量成员
- 引用方式: **结构体变量名.成员名**
- 可以将一个结构体变量**赋值**给另一个结构体变量
- 如 `stu1 = stu2` ; 但 `if(stu1 == stu2)` 错

例

```
struct student
```

```
{    int num;
```

```
    char name[20];
```

```
    char sex;
```

```
    int age;
```

```
    float score;
```

```
    char addr[30];
```

```
}stu1,stu2;
```

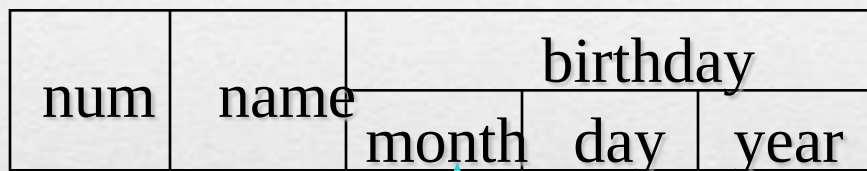
`stu1.num=10;`

`stu1.score=85.5;`

`stu1.score+=stu2.score;`
`stu1.age++;`

结构体变量的引用

- 结构体嵌套时逐级引用



stu1.birthday.month=12;

```
例 struct student
{   int num;
    char name[20];
    struct date
    {   int month;
        int day;
        int year;
    } birthday;
} stu1, stu2;
```


结构体的内存占用

- 一个结构变量的成员变量在内存中是相邻的
- 整个结构变量的将占用多少内存呢？
 - 是所有成员变量的内存总和吗？
 - 我们可以用`sizeof`来获得结构的大小
- `sizeof`是一个C语言的关键字，并不是函数
- 可以用两种形式使用
 - `sizeof (表达式)`
 - 一般都使用`sizeof (变量名)`
 - `sizeof (类型)`
- 求出的结果为表达式值所属类型或者类型占用的字节数

结构体的内存占用

```
#include <stdio.h>
struct student
{
    int num;
    char sex;
}stu1;

int main()
{
    printf("%lu,%lu\n",sizeof(struct student),sizeof(stu1));
}
```

- 事实上，结构所占的实际空间一般是按照机器字长对齐的
 - 不同的编译器、不同的平台，对齐方式会有变化，不过一般的编译器都可以设定按照多大对齐

练习：结构体成员使用

定义如下结构体，依次输入每个结构体成员的值，并输出。

```
struct horse
{
    int age;
    int height;
    char name[20];
    char father[20];
    char mother[20];
};
```

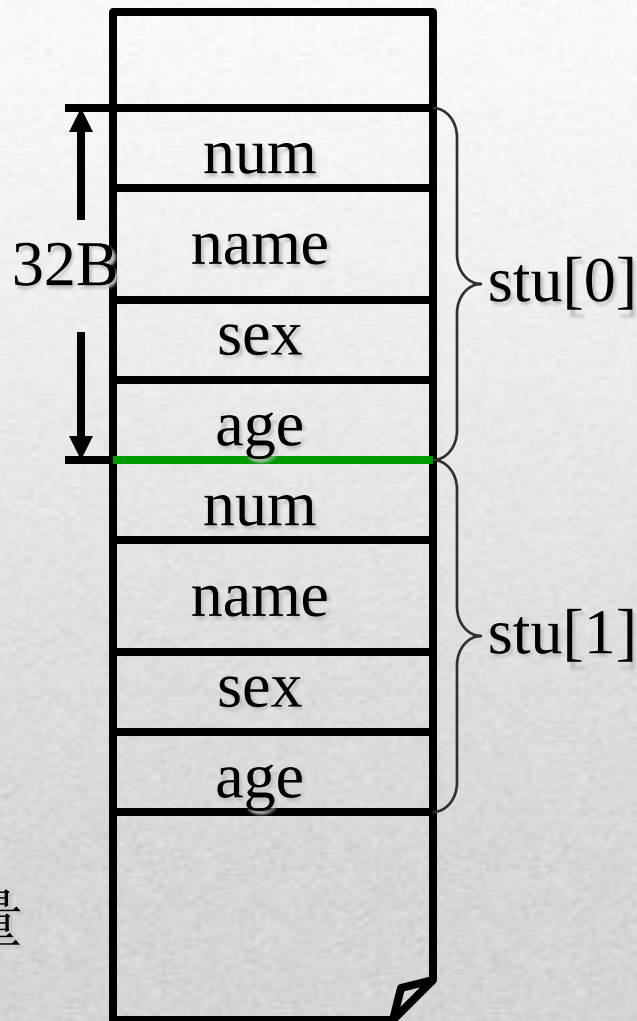
8.3 结构体数组

- 结构体数组的定义
三种形式：

形式一：

```
struct student
{
    int num;
    char name[20];
    char sex;
    int age;
};
struct student stu[2];
```

先定义结构体类型，再定义结构体变量



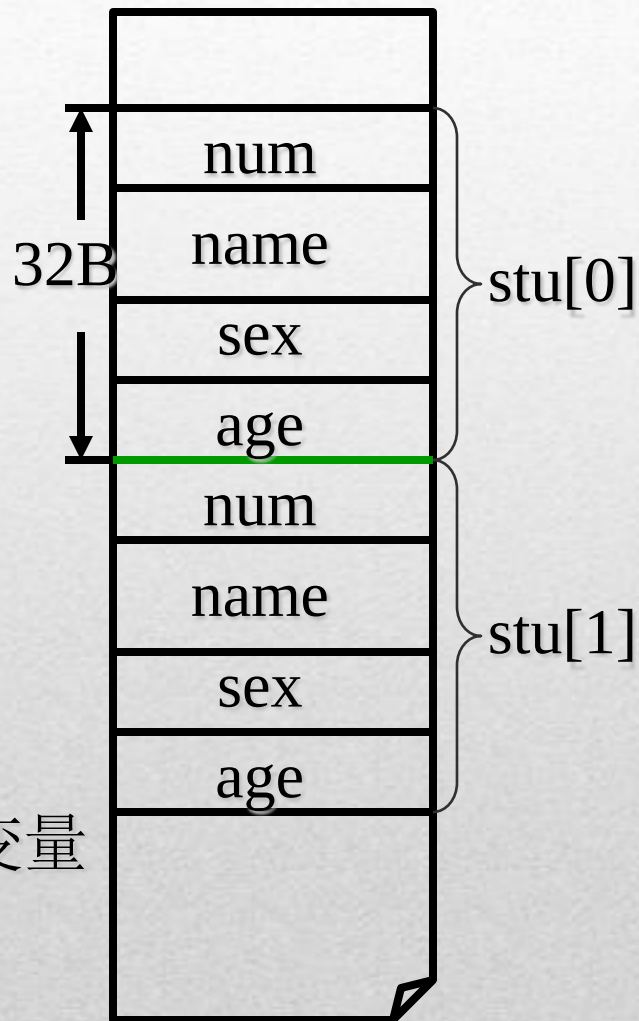
8.3 结构体数组

- 结构体数组的定义
三种形式：

形式二：

```
struct student
{
    int num;
    char name[20];
    char sex;
    int age;
}stu[2];
```

定义结构体类型的同时，定义结构体变量



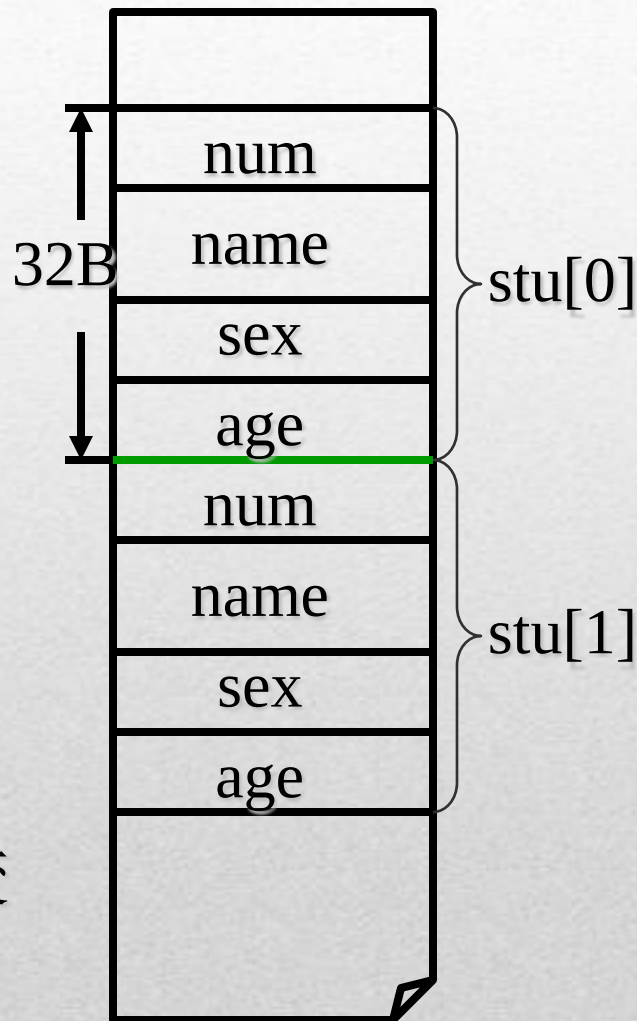
8.3 结构体数组

- 结构体数组的定义
三种形式：

形式三：

```
struct
{
    int num;
    char name[20];
    char sex;
    int age;
}stu[2];
```

定义结构体类型的同时，定义结构体变量，但省略结构体类型名称



结构体数组成员引用

- 引用方式： 结构体数组名[下标].成员名

```
struct student
{
    int num;
    char name[20];
    char sex;
    int age;
}stu[3];
```

```
stu[1].age++;
```

```
strcpy(stu[0].name,"ZhaoDa");
```

结构体数组成员初始化

顺序初始化:

```
struct student
{
    int num;
    char name[20];
    char sex;
    int age;
};
struct student stu[ ]={100,"Wang Lin",'M',20,
                        101,"Li Gang",'M',19,
                        110,"Liu Yan",'F',19};
```


结构体数组成员初始化

分行初始化:

```
struct student
{
    int num;
    char name[20];
    char sex;
    int age;
};
struct student stu[ ]={{100,"Wang Lin",'M',20},
                        {101,"Li Gang",'M',19},
                        {110,"Liu Yan",'F',19}};
```

全部初始化时维数可省

练习：结构体数组成员使用

定义如下结构体，根据用户输入的选择，依次输入每个结构体成员的值（最多可输入50匹马的信息），并输出。

```
struct horse
{
    int age;
    int height;
    char name[20];
    char father[20];
    char mother[20];
};
```

8.4 结构体指针

- 指向结构体变量的指针
 - 定义形式: struct 结构体名 *结构体指针名;

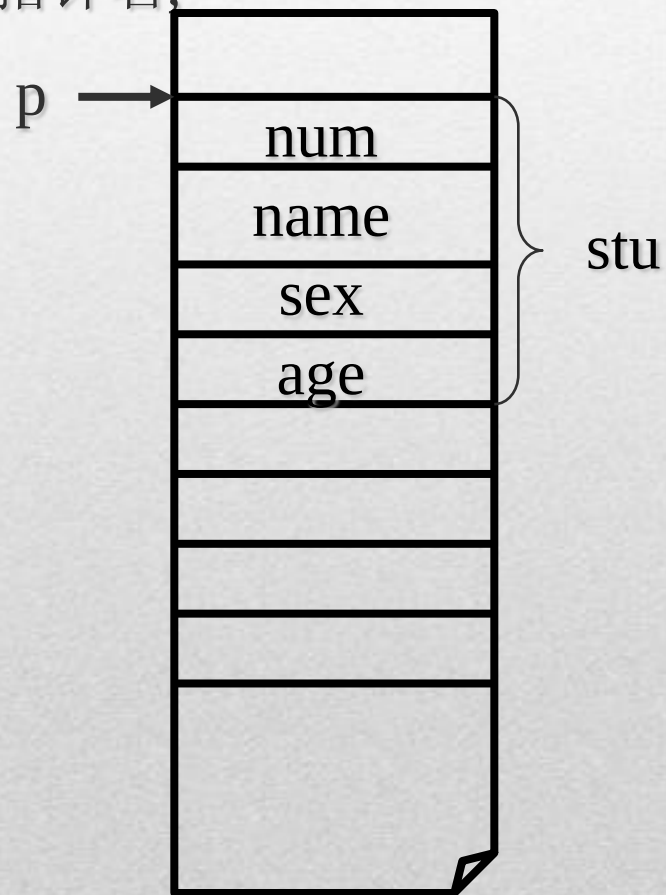
例 struct student *p;

- 使用结构体指针变量引用成员形式

- (*结构体指针名).成员名
- 结构体指针名->成员名
- 结构体变量名.成员名

```
例  int  n;  
    int  *p=&n;  
    *p=10;    ⇔  n=10
```

```
struct  student  stu1;  
struct  student  *p=&stu1;  
stu1.num=101;    ⇔  (*p).num=101
```



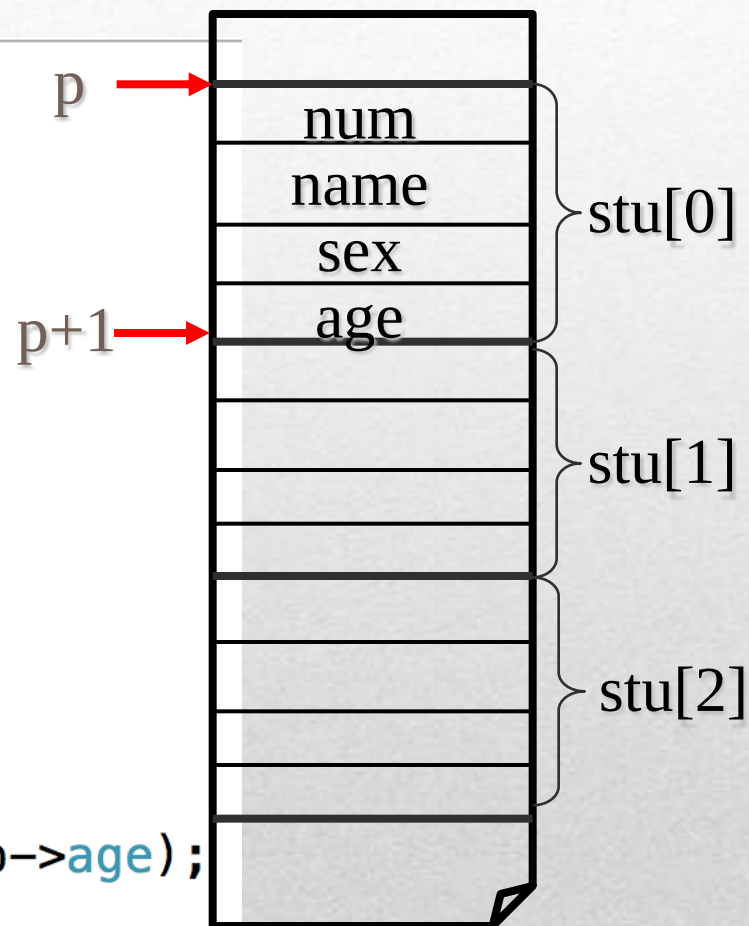
结构体指针

```
#include <stdio.h>
int main()
{
    struct student
    {
        long int num;
        char name[20];
        char sex;
        float score;
    }stu_1,*p;

    p=&stu_1;
    stu_1.num=89101;
    strcpy(stu_1.name,"Li Lin");
    p->sex='M';
    p->score=89.5;
    printf("%d %s %c %d",
           (*p).num,p->name,stu_1.sex,p->score);
}
```


指向结构体数组的指针

```
#include <stdio.h>
struct student
{
    int num;
    char name[20];
    char sex;
    int age;
}stu[3]={10101,"Li Lin",'M',18},
        {10102,"Zhang Fun",'M',19},
        {10104,"Wang Min",'F',20}};
int main()
{
    struct student *p;
    for(p=stu;p<stu+3;p++)
        printf("%d %s %c %d",
                p->num,p->name,p->sex,p->age);
}
```



练习：结构体指针成员使用

定义如下结构体，根据用户输入的选择，依次输入每个结构体成员的值（最多可输入50匹马的信息），每次动态分配存储空间，并输出。

```
struct horse
{
    int age;
    int height;
    char name[20];
    char father[20];
    char mother[20];
};
struct horse *phorse[50];
```

8.5 结构体作函数参数传递

- 用结构体变量作参数----多值传递，直观易懂，效率低
- 用指向结构体变量的指针或数组作参数----开销小，效率高

```

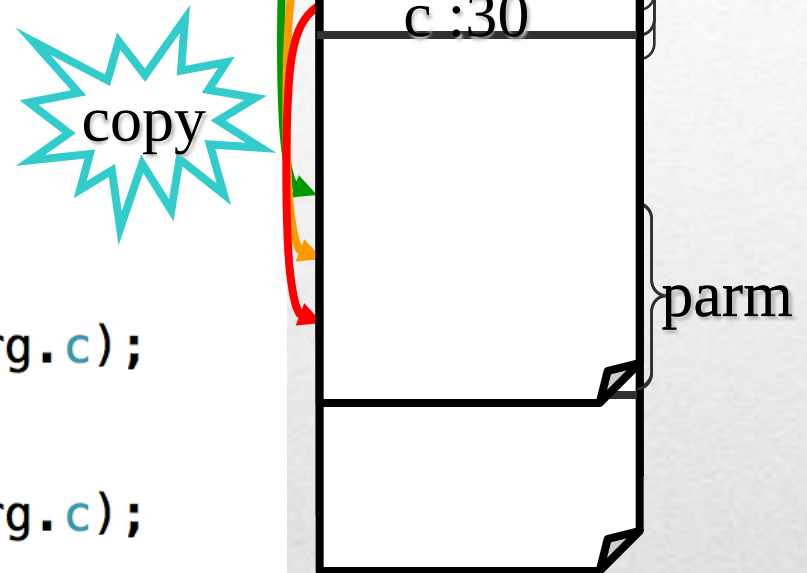
#include <stdio.h>
struct data
{   int a, b, c; };

void func(struct data);

int main()
{   struct data arg;
    arg.a=27;   arg.b=3;
    arg.c=arg.a+arg.b;
    printf("%d%d%d", arg.a, arg.b, arg.c);
    printf("Call Func()....\n");
    func(arg);
    printf("%d%d%d", arg.a, arg.b, arg.c);
}

void func(struct data parm)
{
    printf("%d%d%d", parm.a, parm.b, parm.c);
    printf("Process...\n");
    parm.a=18;    parm.b=5;
    parm.c=parm.a*parm.b;
    printf("%d%d%d", parm.a, parm.b, parm.c);
    printf("Return...\n");
}

```




```

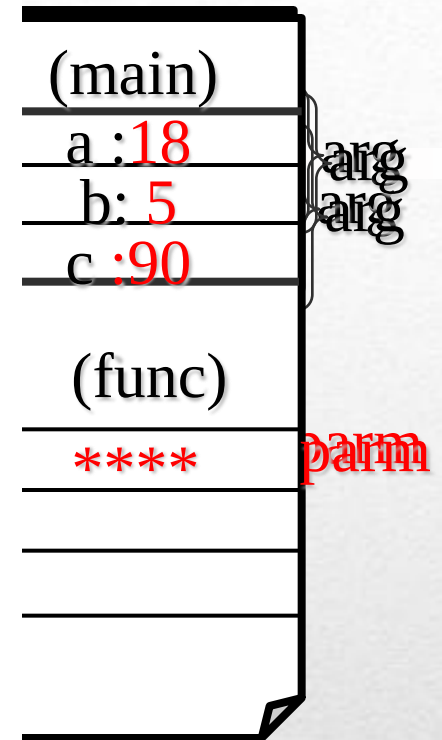
#include <stdio.h>
struct data
{   int a, b, c; };

void func(struct data *parm);

int main()
{   struct data arg;
    arg.a=27;   arg.b=3;
    arg.c=arg.a+arg.b;
    printf("%d%d%d", arg.a, arg.b, arg.c);
    printf("Call Func()....\n");
    func(&arg);
    printf("%d%d%d", arg.a, arg.b, arg.c);
}

void func(struct data *parm)
{
    printf("%d%d%d", parm->a, parm->b, parm->c);
    printf("Process...\n");
    parm->a=18;   parm->b=5;
    parm->c=parm->a*parm->b;
    printf("%d%d%d", parm->a, parm->b, parm->c);
    printf("Return...\n");
}

```



8.6 用typedef为类型重新命名

- 功能：用自定义名字为**已有**数据类型命名

★ **类型定义**简单形式：`typedef type name;`

例 `typedef int INTEGER;`

例 `typedef float REAL;`

例 `INTEGER a,b,c;`
`REAL f1,f2;`



`int a,b,c;`
`float f1,f2;`

说明：

1. `typedef` **没有创造**新数据类型
2. `typedef` **是定义类型**, **不能定义变量**

typedef定义类型步骤

- ① 按定义变量方法先写出定义体 如 `int i;`
- ② 将变量名换成新类型名 如 `int INTEGER;`
- ③ 最前面加typedef 如 `typedef int INTEGER;`
- ④ 用新类型名定义变量 如 `INTEGER i,j;`

例 定义数组类型

- ① `int a[100];`
- ② `int ARRAY[100];`
- ③ `typedef int ARRAY[100];`
- ④ `ARRAY a,b,c;`

⇔ `int a[100],b[100],c[100];`

typedef定义类型步骤

- ① 按定义变量方法先写出定义体 如 `int i;`
- ② 将变量名换成新类型名 如 `int INTEGER;`
- ③ 最前面加typedef 如 `typedef int INTEGER;`
- ④ 用新类型名定义变量 如 `INTEGER i,j;`

例 定义结构体类型

```
① struct date
{
    int month;
    int day;
    int year;
}d;
```

例 定义结构体类型

```
② struct date
{
    int month;
    int day;
    int year;
}DATE;
```

例 定义结构体类型

```
③ typedef struct date
{
    int month;
    int day;
```

```
⇒ struct date
{
    int month;
    int day;
    int year;
}birthday, *p;
```

例 定义结构体类型

```
④ DATE birthday, *p;
```


8.7 位运算

- 指针运算和位运算很适合于编写系统软件的需要，是C语言的重要特色。在计算机用于检测和控制领域中要用到位运算的知识。
- 所谓位运算是指进行二进制位的运算。在系统软件中，常要处理二进制位的问题。例如，将一个存储单元中的各二进制位左移或右移一位，两个数按位相加等。
- C语言提供位运算的功能，与其他高级语言相比，它显然具有很大的优越性。

位运算符和位运算

C语言提供如下表所列出的位运算符

运算符	含义	运算符	含义
&	按位与	~	取反
	按位或	<<	左移
^	按位异或	>>	右移

说明：

- (1)位运算符中除 ~ 以外，均为二目(元)运算符，即要求两侧各有一个运算量。
- (2)运算量只能是整型或字符型的数据，不能为实型数据。

“按位与”运算符(&)

- 参加运算的两个数据，按二进制位进行“与”运算。如果两个相应的二进制位都为1，则该位的结果值为1；否则为0。即

$$0 \& 0 = 0$$

$$0 \& 1 = 0$$

$$1 \& 0 = 0$$

$$1 \& 1 = 1$$

按位与有一些特殊的用途：

(1)清零。

- 如果想将一个单元清零，即使其全部二进制位为0，只要找一个二进制数，其中各个位符合以下条件：原来的数中为1的位，新数中相应位为0。然后使二者进行&运算，即可以达到清零目的。

(2)取一个数中某些指定位。

- 如有一个整数a(2个字节)，想要其中的低字节。只需将a与 $(377)_8$ 按位与即可。

(3)要想将哪一位保留下来

- 就与一个数进行&运算，此数在该位取1。

“按位或”运算符(|)

两个相应的二进制位中只要有一个为1，该位的结果值为1。即

$$0|0=0$$

$$0|1=1$$

$$1|0=1$$

$$1|1=1$$

按位或运算常用来对一个数据的某些位定值为1。

“异或”运算符(^)

异或运算符 $\wedge$ 也称XOR运算符。它的规则是：若参加运算的两个二进制位同号，则结果为0(假)；异号则为1(真)。即

$$0 \wedge 0 = 0$$

$$0 \wedge 1 = 1$$

$$1 \wedge 0 = 1$$

$$1 \wedge 1 = 0$$

“异或”的意思是判断两个相应的位值是否为“异”，为“异”(值不同)就取值(1)；否则为假(0)。

说明^运算符的应用：

(1)与1相^，使特定位翻转

(2)与0相^，保留原值

(3)交换两个值，不用临时变量

$a = a \wedge b;$

$b = b \wedge a;$

$a = a \wedge b;$

“取反”运算符($\sim$)

$\sim$ 是一个单目(元)运算符，用来对一个二进制数按位取反，即将0变1，将1变0。

左移运算符(<<)

用来将一个数的各二进制位全部左移若干位。高位左移后溢出，舍弃。

左移1位相当于该数乘以2。左移比乘法运算快得多，有些C编译程序自动将乘2的运算用左移一位来实现，将称 2^n 的幂运算处理为左移n位。

右移运算符>>

用来将一个数的各二进制位全部右移若干位，移到右端的低位被舍弃。

在右移时，需要注意符号位问题。对无符号数，右移时左边高位移入0；对于有符号的值，如果原来符号位为0(该数为正)，则左边也是移入0。如果符号位原来为1(即负数)，则左边移入0还是1，要取决于所用的计算机系统。有的系统移入0，有的系统移入1。移入0的称为“逻辑右移”，即简单右移；移入1的称为“算术右移”。

位运算赋值运算符

位运算符与赋值运算符可以组成复合赋值运算符，例如： $\&=$ ， $|=$ ， $>>=$ ， $<<=$ ， $\wedge=$ 等。

不同长度的数据进行位运算

如果两个数据长度不同(例如long型和int型)进行位运算时(如 $a \& b$ ，而a为long型，b为int型)，系统会将二者按右端对齐。如果b为正数，则左侧16位补满0；若b为负数，左端应补满1；如果b为无符号整数型，则左端填满0。

8.8 枚举类型

- 实际问题中，有些变量的取值被限制在一定范围内。例如，一个星期内只有七天，一年只有十二个月，一个班每周有六门课程等。
- C语言提供了一种枚举（Enum）类型，可以列出所有可能的取值。
- enum 变量名{ 枚举值列表 };
 - 例如，enum week{sun, mon, tue, wed, thu, fri, sat};
- 枚举变量可以先定义后说明。
 - enum week a, b, c;
- 或者在定义的同时说明：
 - enum week{sun, mon, tue, wed, thu, fri, sat} a, b, c;

```
1  /* Fig. 10.18: fig10_18.c
2     Using an enumeration type */
3  #include <stdio.h>
4
5  /* enumeration constants represent months of the year */
6  enum months {
7     JAN = 1, FEB, MAR, APR, MAY, JUN, JUL, AUG, SEP, OCT, NOV, DEC };
8
9  int main( void )
10 {
11     enum months month; /* can contain any of the 12 months */
12
13     /* initialize array of pointers */
14     const char *monthName[] = { "", "January", "February", "March",
15     "April", "May", "June", "July", "August", "September",
16     "October", "November", "December" };
17
18     /* loop through months */
19     for ( month = JAN; month <= DEC; month++ ) {
20         printf( "%2d%11s\n", month, monthName[ month ] );
21     } /* end for */
22
23     return 0; /* indicates successful termination */
24 } /* end main */
```